

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Вычислительная математика»
Тема: Метод простых итераций

Студент гр. 4342

Озернов Е.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Цель лабораторной работы — изучить численное решение нелинейного уравнения $f(x) = 0$ методом простых итераций. В ходе работы требуется отделить корень, преобразовать исходное уравнение к эквивалентному виду $x = \varphi(x)$, выбрать начальное приближение и реализовать вычисление функции $\varphi(x)$, её производной $\varphi'(x)$ и алгоритма итерационного процесса. Далее необходимо найти корень с заданной точностью ε , исследовать зависимость числа итераций от ε при изменении точности в заданном диапазоне и построить соответствующий график. Дополнительно изучается чувствительность метода к ошибкам исходных данных путём моделирования округления вычислений с параметром Δ .

Задание

Метод простых итераций решения уравнения $f(x) = 0$ состоит в замене исходного уравнения эквивалентным ему уравнением $x = \varphi(x)$ и построении последовательности $x_{n+1} = \varphi(x_n)$, сходящейся при $n \rightarrow \infty$ к точному решению. Достаточные условия сходимости метода простых итераций формулируются теоремой, приведенной [1,2,7].

Рассмотрим один шаг итерационного процесса. Исходя из найденного на предыдущем шаге значения x_{n-1} , вычисляется $y = \varphi(x_{n-1})$. Если $|y - x_{n-1}| > \varepsilon$, то полагается $x_n = y$ и выполняется очередная итерация. Если же $|y - x_{n-1}| < \varepsilon$, то вычисления заканчиваются и за приближенное значение корня принимается величина $x_n = y$. Погрешность результата вычислений зависит от знака производной $\varphi'(x)$: при $\varphi'(x) > 0$ погрешность определения корня составляет $q\varepsilon/(1-q)$, а при $\varphi'(x) < 0$ погрешность не превышает ε . Здесь q - число, такое, что $|\varphi'(x)| \leq q < 1$ на отрезке $[a, b]$. Существование числа q является условием сходимости метода в соответствии с отмеченной выше теоремой.

Для применения метода простых итераций определяющее значение имеет выбор функции $\varphi(x)$ в уравнении $x = \varphi(x)$, эквивалентном исходному. Функцию $\varphi(x)$ необходимо подбирать так, чтобы $|\varphi'(x)| \leq q < 1$. Это обуславливается тем, что если $x < 0$ на отрезке $[a, b]$, то последовательные приближения $x_n = \varphi(x_{n-1})$ будут колебаться около корня s , если же $\varphi'(x) > 0$, то последовательные приближения будут сходиться к корню s монотонно. Следует также помнить, что скорость сходимости последовательности $\{x_n\}$ к корню s функции $f(x)$ тем выше, чем выше число q .

В лабораторной работе № 6 предлагается, используя программы-функции ITER и Round из файла methods.cpp (файл заголовков methods.h, директория LIBR1), найти корень уравнения $f(x) = 0$ с заданной точностью Eps методом итераций, исследовать скорость сходимости и обусловленность метода.

Для данной работы вид функции $f(x)$ задается индивидуально каждому студенту преподавателем из числа вариантов, приведенных в подразделе 3.6.

Порядок выполнения лабораторной работы №6 должен быть следующим.

1) Графически или аналитически отделить корень уравнения $f(x) = 0$.

2) Преобразовать уравнение $f(x) = 0$ к виду $x = \varphi(x)$ так, чтобы в некоторой окрестности $[Left, Right]$ корня производная $\varphi'(x)$ удовлетворяла условию $|\varphi'(x)| \leq q < 1$. При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.

3) Выбрать начальное приближение, лежащее на $[Left, Right]$.

4) Составить подпрограмму для вычисления значений $\varphi(x)$, $\varphi'(x)$, предусмотрев округление вычисленных значений с точностью Δ .

5) Составить главную программу, вычисляющую корень уравнения и содержащую обращение к программам $\varphi(x)$, $\varphi'(x)$ и ITER и индикацию результатов.

6) Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

Вариант 16: $f(x) = \arcsin(2x/(1+x^2)) - e^{(-x^2)}$

Выполнение работы

1. Отделение (изоляция) корня уравнения $f(x) = 0$

Рассматривается функция:

$$f(x) = \arcsin(2x / (1 + x^2)) - e^{(-x^2)}.$$

Функция $f(x)$ непрерывна на всей числовой оси $\mathbb{R}$, так как выражение $2x / (1 + x^2)$ при любых x принимает значения из отрезка $[-1; 1]$, следовательно, функция $\arcsin(2x / (1 + x^2))$ определена и непрерывна. Функция $e^{(-x^2)}$ также непрерывна при всех x . Поэтому разность этих функций непрерывна на $\mathbb{R}$.

Для отделения корня используем теорему Больцано-Коши: если функция $f(x)$ непрерывна на отрезке $[Left; Right]$ и значения $f(Left)$ и $f(Right)$ имеют разные знаки, то на интервале $(Left; Right)$ существует хотя бы один корень уравнения $f(x) = 0$.

Проверим значения функции на концах выбранного отрезка.

При $x = 0$:

$$f(0) = \arcsin(0) - e^0 = 0 - 1 = -1 < 0.$$

При $x = 0.5$:

$$f(0.5) = \arcsin(2 * 0.5 / (1 + 0.5^2)) - e^{(-0.5^2)} = \arcsin(1 / 1.25) - e^{(-0.25)} = \arcsin(0.8) - e^{(-0.25)} \approx 0.9273 - 0.7788 \approx 0.1485 > 0.$$

Так как $f(0) < 0$, $f(0.5) > 0$, то выполняется условие $f(0) * f(0.5) < 0$.

Следовательно, на отрезке $[0; 0.5]$ существует хотя бы один корень уравнения $f(x) = 0$.

Таким образом, в качестве начального отрезка локализации корня можно принять: $[Left; Right] = [0; 0.5]$.

2. Преобразование уравнения $f(x) = 0$ к виду $x = \varphi(x)$

Исходное уравнение имеет вид $f(x) = \arcsin(2x / (1 + x^2)) - e^{(-x^2)} = 0$. Для применения метода простых итераций необходимо преобразовать его к эквивалентному виду $x = \varphi(x)$ так, чтобы на выбранном отрезке $[Left; Right]$ выполнялось условие сходимости $|\varphi'(x)| \leq q < 1$. В качестве рабочего отрезка выбираем $[Left; Right] = [0.4; 0.5]$. На этом отрезке $x > 0$ и $x < 1$, поэтому можно

использовать тождество $\arcsin(2x / (1 + x^2)) = 2 \arctan(x)$. Тогда исходное уравнение $\arcsin(2x / (1 + x^2)) - e^{(-x^2)} = 0$ на отрезке $[0.4; 0.5]$ эквивалентно уравнению $2 \arctan(x) - e^{(-x^2)} = 0$, то есть $2 \arctan(x) = e^{(-x^2)}$. Разделив обе части на 2, получаем $\arctan(x) = e^{(-x^2)} / 2$. Применяя к обеим частям функцию $\tan$, приходим к уравнению $x = \tan(e^{(-x^2)} / 2)$. Таким образом, в качестве функции итераций выбираем $\varphi(x) = \tan(e^{(-x^2)} / 2)$.

Теперь проверим условие сходимости. Найдём производную функции $\varphi(x)$. Пусть $\varphi(x) = \tan(u)$, где $u = e^{(-x^2)} / 2$. Тогда $\varphi'(x) = (1 / \cos^2(u)) * u'$. Поскольку $u' = (1/2) * (e^{(-x^2)})' = (1/2) * (-2x * e^{(-x^2)}) = -x * e^{(-x^2)}$, получаем $\varphi'(x) = -x * e^{(-x^2)} / \cos^2(e^{(-x^2)} / 2)$. Численная оценка на отрезке $[0.4; 0.5]$ показывает, что $|\varphi'(x)| \leq q$, где $q \approx 0.455 < 1$. Следовательно, на данном отрезке выполнено достаточное условие сходимости метода простых итераций

Кроме того, функция $\varphi(x)$ переводит отрезок $[0.4; 0.5]$ в себя: $\varphi(0.4) = \tan(e^{(-0.16)} / 2) \approx 0.4539$, а $\varphi(0.5) = \tan(e^{(-0.25)} / 2) \approx 0.4104$, и оба значения принадлежат отрезку $[0.4; 0.5]$. Это подтверждает корректность выбора функции $\varphi(x)$. Так как $q < 1$, итерационный процесс $x(n+1) = \varphi(x(n))$ сходится к корню уравнения. Поскольку производная $\varphi'(x)$ на данном отрезке отрицательна, последовательные приближения будут сходиться к корню колебательно. Таким образом, для решения уравнения методом простых итераций принимаем преобразование $x = \varphi(x) = \tan(e^{(-x^2)} / 2)$, которое удовлетворяет условию $|\varphi'(x)| \leq q < 1$ на отрезке $[0.4; 0.5]$.

3. Выбор начального приближения

В качестве начального приближения необходимо выбрать значение x_0 , принадлежащее отрезку $[Left; Right]$, на котором выполняются условия сходимости метода простых итераций. В данной работе в качестве рабочего интервала выбран отрезок $[0.4; 0.5]$, поэтому начальное приближение должно удовлетворять условию $x_0 \in [0.4; 0.5]$. Удобно взять начальное значение в середине этого отрезка, то есть $x_0 = (Left + Right) / 2 = (0.4 + 0.5) / 2 = 0.45$. Такое значение принадлежит выбранному интервалу и является естественным начальным приближением, так как расположено близко к искомому корню.

Следовательно, в данной работе в качестве начального приближения принимается $x_0 = 0.45$.

4. Составление подпрограммы для вычисления значений $\varphi(x)$ и $\varphi'(x)$ с округлением до точности Delta

Для реализации метода простых итераций необходимо составить подпрограммы, которые вычисляют значение функции итераций $\varphi(x)$ и её производной $\varphi'(x)$, так как именно эти величины используются в основном алгоритме. Функция $\varphi(x)$ нужна для построения последовательности приближений $x_{(n+1)} = \varphi(x_{(n)})$, а производная $\varphi'(x)$ требуется для проверки условия сходимости метода, то есть условия $|\varphi'(x)| \leq q < 1$ на выбранном отрезке $[Left; Right]$. Кроме того, по условию лабораторной работы необходимо предусмотреть округление вычисляемых значений с точностью Delta, чтобы затем можно было исследовать влияние ошибок округления на результат.

В данной работе после преобразования исходного уравнения используется функция $\varphi(x) = \tan(e^{-x^2} / 2)$. Чтобы получить её производную, воспользуемся правилом дифференцирования сложной функции. Пусть $u(x) = e^{-x^2} / 2$, тогда $\varphi(x) = \tan(u(x))$. Производная функции $\tan(u)$ равна $1 / \cos^2(u)$, умноженному на производную внутренней функции $u'(x)$. Найдём $u'(x)$: $u'(x) = (1/2) * (e^{-x^2})' = (1/2) * (-2x * e^{-x^2}) = -x * e^{-x^2}$. Следовательно, производная функции итераций имеет вид $\varphi'(x) = -x * e^{-x^2} / \cos^2(e^{-x^2} / 2)$. Именно эта формула и используется в программе.

Для моделирования округления вводится отдельная подпрограмма `Round(value, Delta)`. Её назначение состоит в том, чтобы округлить число `value` с шагом `Delta`. Если `Delta = 0`, то округление не выполняется, и число возвращается без изменений. Это соответствует вычислениям без учёта ошибок округления. Если же `Delta > 0`, то число округляется до ближайшего значения, кратного `Delta`. Например, при `Delta = 0.001` число `0.437906` округлится до `0.438`, а число `0.4372` — до `0.437`. Такое округление позволяет искусственно ввести ограниченную точность вычислений и посмотреть, как она влияет на найденный корень.

Подпрограмма $\text{Round}(\text{value}, \Delta)$ в программе реализуется по формуле $\text{Round}(\text{value}, \Delta) = \text{round}(\text{value} / \Delta) * \Delta$. Здесь round — стандартная операция округления до ближайшего целого. Смысл этой формулы в том, что сначала число переводится в количество шагов Δ , затем это количество округляется до ближайшего целого, после чего выполняется обратный переход к исходному масштабу. Если $\Delta \leq 0$, то подпрограмма просто возвращает исходное значение.

После этого составляется подпрограмма $\phi(x, \Delta)$ для вычисления функции $\phi(x)$. Логика её работы следующая. Сначала входное значение x при необходимости округляется с помощью $\text{Round}(x, \Delta)$. Это делается для того, чтобы моделировать ситуацию, когда и аргумент функции известен лишь с некоторой ограниченной точностью. Затем по формуле $\phi(x) = \tan(e^{(-x^2)} / 2)$ вычисляется значение функции. После этого полученное значение также округляется с помощью $\text{Round}(\text{value}, \Delta)$. Таким образом, округление применяется и к аргументу, и к результату вычисления функции. Это соответствует реальным вычислениям на машине, где промежуточные данные и конечный результат хранятся с ограниченной точностью.

Аналогично строится подпрограмма $d\phi(x, \Delta)$ для вычисления производной $\phi'(x)$. Сначала значение x округляется до точности Δ , затем по формуле $\phi'(x) = -x * e^{(-x^2)} / \cos^2(e^{(-x^2)} / 2)$ вычисляется производная, и после этого результат также округляется с помощью функции Round . Подпрограмма $d\phi(x, \Delta)$ используется в программе при оценке величины $q = \max |\phi'(x)|$ на рабочем отрезке $[\text{Left}; \text{Right}]$. Если оказывается, что $q < 1$, то это означает выполнение достаточного условия сходимости метода простых итераций.

Таким образом, в программе реализуются три взаимосвязанные подпрограммы. Первая подпрограмма $\text{Round}(\text{value}, \Delta)$ отвечает за округление чисел с заданной точностью. Вторая подпрограмма $\phi(x, \Delta)$ вычисляет значение функции итераций $\phi(x) = \tan(e^{(-x^2)} / 2)$ с учётом округления. Третья подпрограмма $d\phi(x, \Delta)$ вычисляет значение производной $\phi'(x) = -x * e^{(-x^2)}$

$\tan(e^{-x^2}) / 2$) также с учётом округления. Наличие этих подпрограмм позволяет не только реализовать сам метод простых итераций, но и выполнить дополнительное исследование обусловленности метода, то есть изучить, насколько сильно ошибки округления влияют на конечное приближённое значение корня.

В программной реализации это выглядит следующим образом. Функция `Round(value, Delta)` проверяет, равно ли `Delta` нулю. Если да, то возвращается исходное значение `value`. Если нет, то выполняется округление по формуле `round(value / Delta) * Delta`. Функция `phi(x, Delta)` сначала вызывает `Round` для аргумента `x`, затем вычисляет значение $\tan(e^{-x^2}) / 2$ и округляет полученный результат. Функция `dphi(x, Delta)` также сначала округляет аргумент `x`, затем вычисляет значение $-x * e^{-x^2} / \cos^2(e^{-x^2}) / 2$ и округляет результат. Благодаря такому построению программы можно запускать вычисления как без округления, так и с различными значениями `Delta`, например 10^{-2} , 10^{-4} , 10^{-6} и так далее, и сравнивать, как меняется найденный корень.

Итак, составление подпрограмм для вычисления $\phi(x)$ и $\phi'(x)$ с округлением до точности `Delta` является необходимым этапом лабораторной работы, так как именно эти подпрограммы обеспечивают выполнение итерационного процесса, проверку условия сходимости и исследование влияния вычислительных погрешностей на результат.

5. Составление головной программы, вычисляющей корень уравнения и содержащей обращение к программам $\phi(x)$, $\phi'(x)$ и ITER, а также индикацию результатов

Головная программа предназначена для организации всего вычислительного процесса метода простых итераций. Именно в ней задаются исходные параметры задачи, вызываются подпрограммы вычисления функции $\phi(x)$, её производной $\phi'(x)$ и основной итерационный алгоритм ITER, а затем выводятся результаты вычислений. Таким образом, головная программа

объединяет все ранее разработанные подпрограммы в единую схему решения задачи.

В начале работы программы задаются основные исходные данные. Прежде всего указывается рабочий интервал, на котором будет применяться метод простых итераций. В данной работе таким интервалом является $[Left; Right] = [0.4; 0.5]$, поскольку именно на нём для выбранной функции $\varphi(x)$ выполняется условие сходимости $|\varphi'(x)| \leq q < 1$. Далее задаётся начальное приближение x_0 , которое выбирается внутри этого интервала. В программе принимается $x_0 = (Left + Right) / 2 = 0.45$. Кроме того, задаются точность вычисления корня Eps и параметр округления $Delta$. Значение Eps определяет условие остановки итерационного процесса, а $Delta$ позволяет при необходимости моделировать округление вычислений.

После задания исходных параметров головная программа должна вывести информацию о рассматриваемой задаче. Обычно указывается вид исходной функции $f(x)$, используемое преобразование $x = \varphi(x)$, выбранный интервал $[Left; Right]$, начальное приближение x_0 , а также значения Eps и $Delta$. Такой вывод позволяет сразу видеть, для какой задачи выполняются вычисления и с какими параметрами запускается программа.

Затем головная программа обращается к подпрограмме $\text{phi}(x, Delta)$, чтобы вычислить значения функции $\varphi(x)$ на концах интервала. Это делается для дополнительной проверки корректности выбора функции итераций. Если значения $\varphi(Left)$ и $\varphi(Right)$ принадлежат отрезку $[Left; Right]$, то это подтверждает, что функция $\varphi(x)$ переводит выбранный отрезок в себя. В нашей задаче эта проверка показывает, что $\varphi(0.4)$ и $\varphi(0.5)$ действительно лежат внутри интервала $[0.4; 0.5]$, следовательно, выбранное преобразование является корректным для метода простых итераций.

После этого головная программа вызывает подпрограмму $\text{dphi}(x, Delta)$ для оценки величины $q = \max |\varphi'(x)|$ на отрезке $[Left; Right]$. Это необходимо для проверки достаточного условия сходимости метода. На практике программа вычисляет значения $|\varphi'(x)|$ в наборе точек выбранного интервала и находит

наибольшее из них. Если полученная оценка q оказывается меньше единицы, то можно сделать вывод о сходимости метода простых итераций. В нашей задаче получается $q \approx 0.455$, то есть $q < 1$, и, следовательно, метод на данном интервале сходится.

После проверки условия сходимости головная программа вызывает основную подпрограмму ITER. Именно эта подпрограмма реализует итерационный процесс $x(n+1) = \varphi(x(n))$. В неё передаются функция $\varphi(x)$, производная $\varphi'(x)$, границы интервала Left и Right, начальное приближение x_0 , требуемая точность Eps, максимально допустимое число итераций и параметр округления Delta. Внутри ITER последовательно вычисляются новые приближения к корню до тех пор, пока не выполнится условие остановки $|x(n+1) - x(n)| < \text{Eps}$. Как только это условие становится истинным, процесс завершается, и найденное значение принимается за приближённый корень уравнения.

После возврата из подпрограммы ITER головная программа получает основные результаты вычислений: найденное приближённое значение корня, число выполненных итераций, значение последней разности $|x(n+1) - x(n)|$ и признак успешной сходимости метода. Эти данные выводятся на экран. Кроме того, программа вычисляет значение исходной функции $f(x)$ в найденной точке, чтобы показать, насколько близко найденное значение к точному корню. Если значение $f(x^*)$ близко к нулю, это подтверждает корректность результата.

Также в головной программе удобно вывести значение производной $\varphi'(x)$ в найденной точке. Это позволяет дополнительно проанализировать характер сходимости метода. В нашей задаче $\varphi'(x) < 0$, поэтому приближения сходятся к корню колебательно, то есть поочерёдно оказываются то больше, то меньше истинного значения корня. Если бы $\varphi'(x)$ было положительным, сходимость была бы монотонной.

Кроме вывода результатов на экран, головная программа сохраняет данные для последующего анализа в файлы. В частности, в отдельный CSV-файл записывается история итераций: номер итерации n , текущее приближение $x(n)$, разность $|x(n) - x(n-1)|$ и, при необходимости, абсолютная ошибка относительно

эталонного значения корня. Эти данные затем используются для построения графика зависимости ошибки от номера итерации, то есть для исследования скорости сходимости метода. В другой CSV-файл записываются результаты серии запусков программы при различных значениях Delta, что позволяет исследовать чувствительность метода к округлению и построить график зависимости ошибки от величины Delta.

Таким образом, головная программа выполняет сразу несколько функций. Во-первых, она задаёт исходные параметры задачи. Во-вторых, она вызывает подпрограммы $\phi(x, \Delta)$ и $d\phi(x, \Delta)$ для проверки корректности выбранного итерационного преобразования и условия сходимости. В-третьих, она запускает подпрограмму ITER, которая непосредственно находит приближённое значение корня. В-четвёртых, она выводит на экран все основные результаты: найденный корень, число итераций, величину невязки $f(x^*)$, значение $\phi'(x^*)$ и оценку погрешности. В-пятых, она формирует данные для последующего построения графиков и исследования скорости сходимости и обусловленности метода.

Следовательно, именно головная программа является центральной частью всей лабораторной работы, так как она объединяет вычисление функции $\phi(x)$, её производной $\phi'(x)$, выполнение метода простых итераций и вывод результатов в единую завершённую систему численного решения уравнения $f(x) = 0$.

6. Проведение вычислений по программе. Исследование скорости сходимости и обусловленности метода

После реализации подпрограмм $\phi(x)$, $d\phi(x)$ и ITER были проведены вычисления для уравнения $f(x) = \arcsin(2x / (1 + x^2)) - e^{(-x^2)} = 0$ методом простых итераций. В программе автоматически был найден рабочий интервал от деления корня $[0.4; 0.5]$, так как на его концах значения функции имеют разные знаки: $f(0.4) = -0.091131034741$, $f(0.5) = 0.148494434930$. Для метода простых итераций использовалась функция $\phi(x) = \tan(e^{(-x^2)} / 2)$. При этом было получено $\phi(0.4) = 0.453875231986$ и $\phi(0.5) = 0.410354165789$, то есть функция $\phi(x)$ переводит отрезок $[0.4; 0.5]$ в себя. Численная оценка максимума $|\phi'(x)|$

на данном отрезке дала значение $q = 0.454971734280$, следовательно, выполняется условие сходимости $|\phi'(x)| \leq q < 1$, и метод простых итераций на выбранном интервале сходится.

В качестве начального приближения было выбрано $x_0 = 0.45$. Для получения эталонного значения корня программа была запущена при очень малой точности $Eps = 1e-14$ и без округления, то есть при $Delta = 0$. В результате было получено значение $x^* = 0.437906525771$, причём проверка показала, что $f(x^*)$ практически равно нулю. Это значение далее использовалось как опорное при исследовании точности вычислений и чувствительности метода к округлению.

При рабочем запуске программы с параметрами $Eps = 1e-8$ и $Delta = 1e-10$ был найден корень $x = 0.437906524400$. Значение исходной функции в найденной точке составило $f(x) = -0.000000003292$, число итераций оказалось равным 19, а последний шаг итерационного процесса $|x(n+1) - x(n)|$ составил 0.000000004500. Абсолютная ошибка относительно эталонного значения корня оказалась равной 0.000000001371. Так как на выбранном интервале $\phi'(x) < 0$, для данного метода выполняется теоретическая оценка погрешности, согласно которой ошибка не превышает Eps . Полученный результат подтверждает корректность работы программы и выполнение условия останова итерационного процесса.

Исследование скорости сходимости метода проводилось по данным файла `iterations_vs_eps_iter.csv`. В этом файле для различных значений точности Eps зафиксированы число итераций, найденный корень и величина последнего шага. По этим данным был построен график зависимости числа итераций от точности, сохранённый в файле `iterations_vs_eps_iter.png`. Анализ содержимого файла `iterations_vs_eps_iter.csv` показывает, что при уменьшении Eps число итераций возрастает. Например, при $Eps = 0.1$ метод завершился за 1 итерацию, при $Eps = 0.01$ потребовалось 2 итерации, при $Eps = 0.001$ — 5 итераций, при $Eps = 1e-6$ — 13 итераций, а при $Eps = 1e-8$ — уже 19 итераций. Таким образом, чем более высокая точность требуется, тем больше шагов нужно выполнить методу простых итераций. Рост числа итераций происходит ступенчато, что объясняется

дискретным условием остановки $|x(n+1) - x(n)| < \text{Eps}$: при плавном уменьшении Eps число итераций меняется не непрерывно, а увеличивается на единицу в те моменты, когда для достижения новой точности становится необходим ещё один шаг.

По данным файла `iterations_vs_eps_iter.csv` и рисунка `iterations_vs_eps_iter.png` можно сделать вывод, что метод обладает устойчивой сходимостью на выбранном интервале. График показывает монотонный рост числа итераций при уменьшении Eps , что соответствует теоретическим ожиданиям для сходящегося итерационного процесса. При этом даже для достаточно строгой точности $\text{Eps} = 1\text{e-}8$ число итераций остаётся умеренным. Следовательно, для рассматриваемой функции и выбранного преобразования $x = \text{phi}(x)$ метод простых итераций работает эффективно.

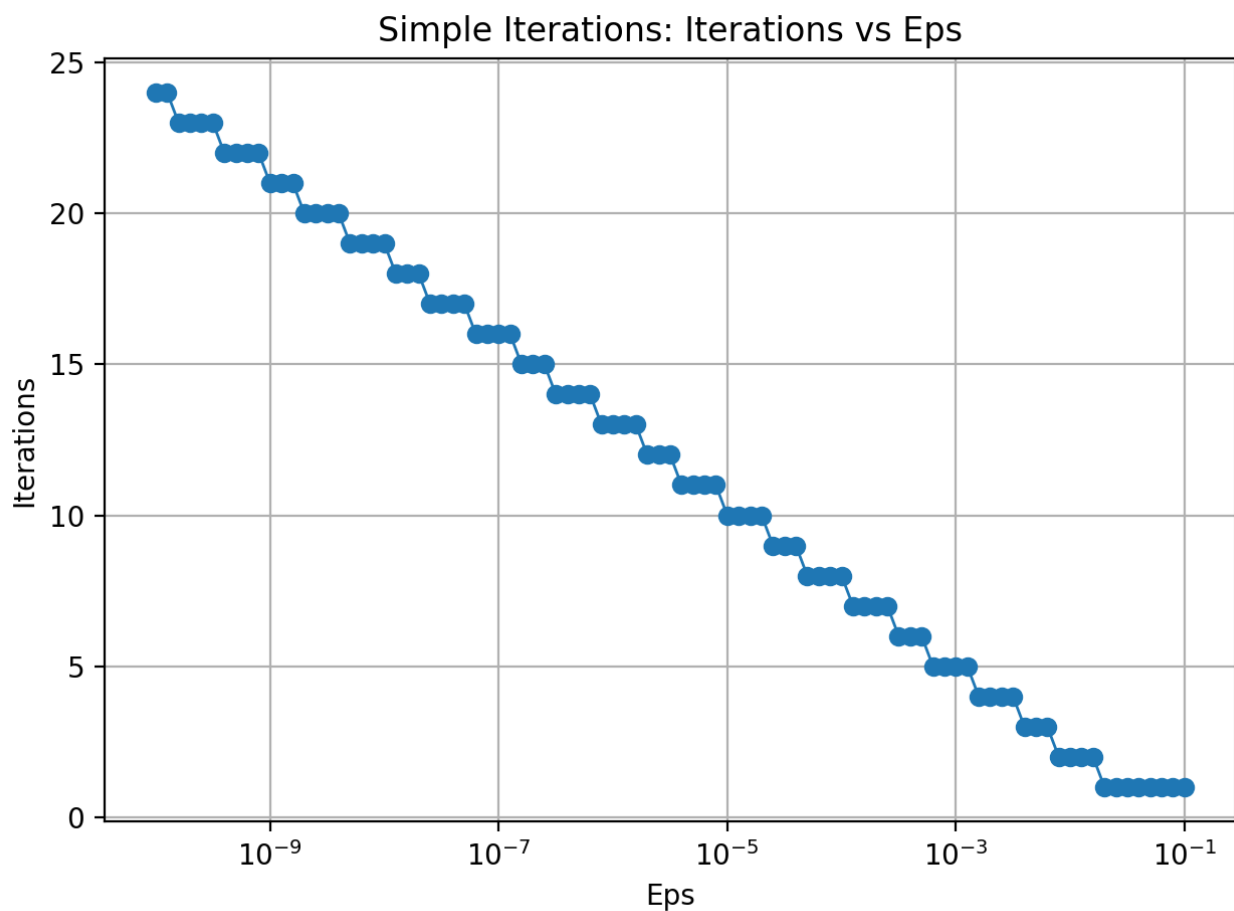


Рисунок 1 — Зависимость числа итераций метода простых итераций от требуемой точности Eps по данным файла iterations_vs_eps_iter.csv.

Исследование обусловленности метода проводилось по данным файла sensitivity_vs_delta_iter.csv. В этом файле собраны результаты вычислений при фиксированной точности $Eps = 1e-6$ и различных значениях параметра округления Delta. Для каждого значения Delta в таблице указаны число итераций, найденное приближение корня и абсолютная ошибка относительно эталонного значения $x^* = 0.437906525771$. По этим данным был построен график зависимости ошибки от Delta, сохранённый в файле sensitivity_vs_delta_iter.png.

Анализ файла sensitivity_vs_delta_iter.csv показывает, что при грубом округлении точность результата заметно ухудшается. Например, при $Delta = 0.01$ найденный корень равен 0.44, а абсолютная ошибка составляет 0.002093474229. При $Delta = 0.001$ найденное значение равно 0.438, а ошибка уменьшается до 0.000093474229. При $Delta = 0.0001$ ошибка становится равной 0.000006525771. При дальнейшем уменьшении Delta до $1e-5$ и $1e-7$ ошибка уже имеет порядок $10^{(-6)}$ - $10^{(-7)}$, то есть существенно уменьшается. Это означает, что с уменьшением шага округления Delta влияние вычислительных погрешностей на итоговый результат ослабевает.

Вместе с тем результаты показывают, что при слишком грубом округлении или при неудачном соотношении между Eps и Delta итерационный процесс может работать нестабильно. Так, при $Delta = 0.1$ программа не достигла условия остановки за максимально допустимое число итераций и фактически потеряла сходимость из-за слишком сильного округления. Аналогичная ситуация наблюдалась и при $Delta = 1e-6$, когда значение Delta оказалось сравнимо с требуемой точностью Eps. Это связано с тем, что округление может исказить обычное поведение итерационной последовательности и привести к заикливанию на нескольких близких значениях вместо нормального приближения к корню. Следовательно, метод простых итераций чувствителен к

погрешностям округления, и для его корректной работы параметр Delta должен быть выбран достаточно малым по сравнению с требуемой точностью Eps.

По данным файла `sensitivity_vs_delta_iter.csv` и рисунка `sensitivity_vs_delta_iter.png` можно сделать вывод, что обусловленность метода в данной задаче является удовлетворительной при достаточно малых значениях Delta. Когда округление невелико, ошибка решения остаётся малой и результат практически совпадает с эталонным корнем. Однако при увеличении Delta влияние округления быстро возрастает, что приводит к заметному ухудшению точности, а в отдельных случаях — к нарушению нормальной сходимости итерационного процесса.

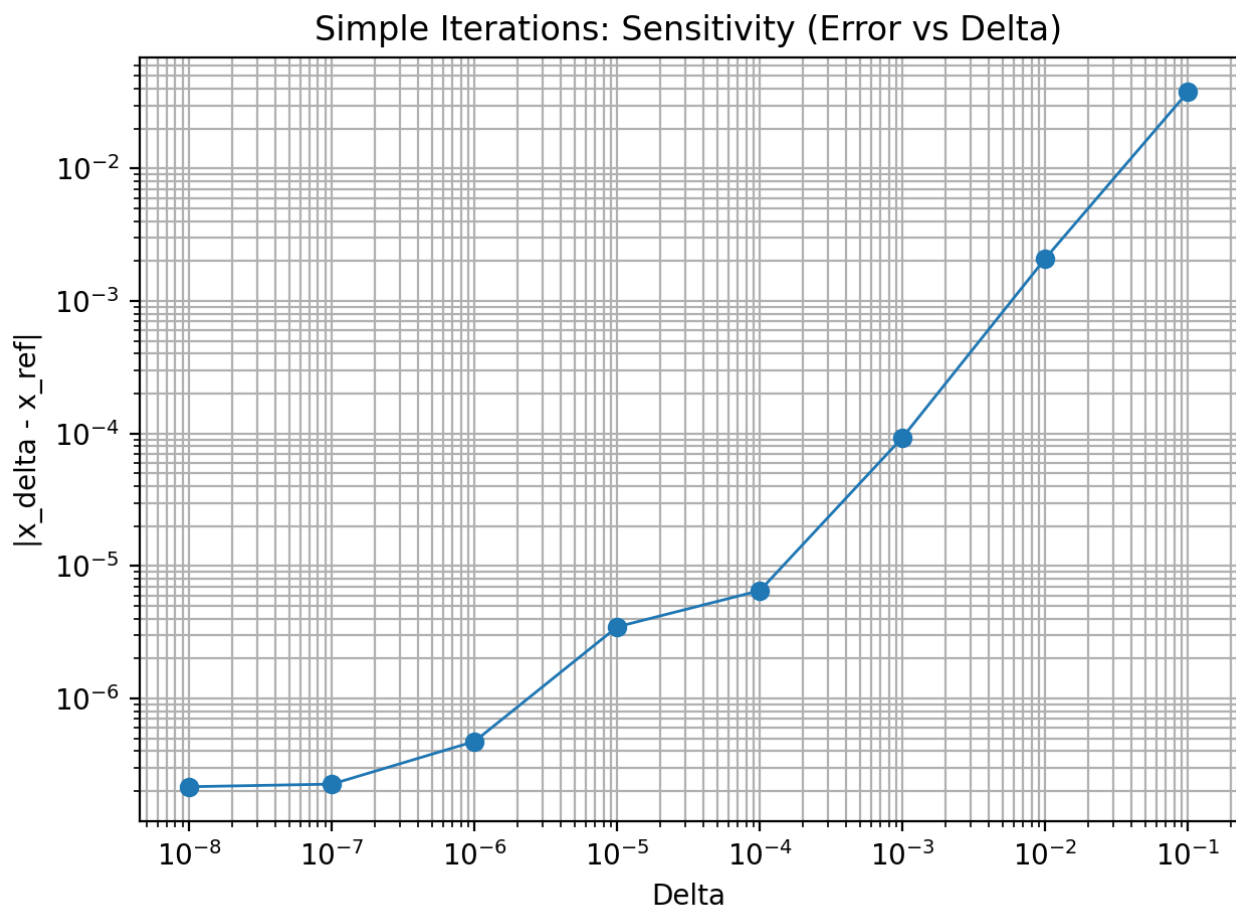


Рисунок 2 — Зависимость абсолютной ошибки найденного корня от параметра округления Delta по данным файла `sensitivity_vs_delta_iter.csv`.

Таким образом, проведённые вычисления показали, что метод простых итераций для уравнения $f(x) = \arcsin(2x / (1 + x^2)) - e^{(-x^2)}$ при выборе функции $\phi(x) = \tan(e^{(-x^2)} / 2)$ успешно сходится на отрезке $[0.4; 0.5]$ и позволяет получить приближённое значение корня $x \approx 0.437906525$. Исследование скорости сходимости показало, что с уменьшением Eps число итераций возрастает, что подтверждается данными файла `iterations_vs_eps_iter.csv` и графиком `iterations_vs_eps_iter.png`. Исследование обусловленности показало, что точность метода заметно зависит от параметра округления Delta: при малых значениях Delta метод даёт точные результаты, а при грубом округлении погрешность возрастает, что подтверждается данными файла `sensitivity_vs_delta_iter.csv` и графиком `sensitivity_vs_delta_iter.png`.

Вывод.

В ходе лабораторной работы был изучен метод простых итераций для решения нелинейного уравнения $f(x) = 0$ на примере функции $f(x) = \arcsin(2x / (1 + x^2)) - e^{(-x^2)}$. Исходное уравнение было преобразовано к виду $x = \phi(x)$, выбран рабочий отрезок $[0.4; 0.5]$ и проверено условие сходимости $|\phi'(x)| \leq q < 1$. В программе были реализованы подпрограммы для вычисления $\phi(x)$, $\phi'(x)$, округления с точностью Δ и основной итерационный алгоритм.

Проведённые вычисления показали, что метод сходится и позволяет найти приближённое значение корня $x \approx 0.437906525$. Исследование скорости сходимости показало, что при уменьшении ϵ_{ps} число итераций возрастает. Исследование обусловленности показало, что при увеличении Δ точность результата ухудшается. Таким образом, цель работы достигнута, а метод простых итераций подтвердил свою эффективность для решения данного уравнения.

ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: methods.h

```
#pragma once
#include <functional>

struct IterationResult {
    double x;           // найденное приближение корня
    int iters;          // число итераций
    bool ok;            // сошлось ли
    double last_step;   //  $|x_n - x_{(n-1)}|$ 
};

double Round(double value, double Delta);

// Оценка  $q = \max |\phi'(x)|$  на [Left; Right]
double estimate_q(const std::function<double(double)>& dphi,
                 double Left, double Right,
                 int samples = 1000,
                 double Delta = 0.0);

// Метод простых итераций:
// phi - функция  $\phi(x)$ 
// dphi - производная  $\phi'(x)$ 
// Left, Right - интервал, где изолирован корень
// x0 - начальное приближение (лежит на [Left;Right])
// Eps - требуемая точность
// nmax - ограничение итераций
// Delta - округление значений phi и phi' через Round (0 => без округления)
IterationResult ITER(const std::function<double(double)>& phi,
                    const std::function<double(double)>& dphi,
                    double Left, double Right,
                    double x0,
                    double Eps,
                    int nmax = 1000000,
                    double Delta = 0.0);
```

Название файла: methods.cpp

```
#include "methods.h"
#include <cmath>
#include <limits>
#include <algorithm>

double Round(double value, double Delta) {
    if (Delta <= 0.0) return value;
    return std::round(value / Delta) * Delta;
}

double estimate_q(const std::function<double(double)>& dphi,
                 double Left, double Right,
                 int samples,
                 double Delta) {
    double q = 0.0;
    for (int i = 0; i <= samples; ++i) {
        double x = Left + (Right - Left) * static_cast<double>(i) /
static_cast<double>(samples);
        double val = Round(dphi(x), Delta);
```

```

        q = std::max(q, std::abs(val));
    }
    return q;
}

IterationResult ITER(const std::function<double(double)>& phi,
                    const std::function<double(double)>& dphi,
                    double Left, double Right,
                    double x0,
                    double Eps,
                    int nmax,
                    double Delta) {
    IterationResult res{};
    res.x = std::numeric_limits<double>::quiet_NaN();
    res.iters = 0;
    res.ok = false;
    res.last_step = std::numeric_limits<double>::infinity();

    // Проверка условия сходимости  $q < 1$ 
    double q = estimate_q(dphi, Left, Right, 2000, Delta);
    if (!(q < 1.0)) {
        return res;
    }

    double x_prev = Round(x0, Delta);

    for (int n = 1; n <= nmax; ++n) {
        double x_next = Round(phi(x_prev), Delta);
        double step = std::abs(x_next - x_prev);

        res.last_step = step;

        // Основной критерий из методички:  $|x_n - x_{(n-1)}| < Eps$ 
        if (step < Eps) {
            res.x = x_next;
            res.iters = n;
            res.ok = true;
            return res;
        }

        // Небольшая защита: если ушли за интервал, считаем, что что-то
        пошло не так
        if (x_next < Left - 1e-12 || x_next > Right + 1e-12) {
            res.x = x_next;
            res.iters = n;
            res.ok = false;
            return res;
        }

        x_prev = x_next;
    }

    res.x = x_prev;
    res.iters = nmax;
    res.ok = false;
    return res;
}

```

Название файла: main.cpp

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <vector>
#include <fstream>
#include <algorithm>
#include "methods.h"

//  $f(x) = \arcsin(2x/(1+x^2)) - e^{-x^2}$ 
double f(double x) {
    double t = (2.0 * x) / (1.0 + x * x);

    // защита от погрешностей double: зажимаем в [-1;1]
    if (t > 1.0) t = 1.0;
    if (t < -1.0) t = -1.0;

    return std::asin(t) - std::exp(-x * x);
}

// Выбранное преобразование:
//  $2 \cdot \arctan(x) = e^{-x^2} \Rightarrow x = \tan(e^{-x^2}/2)$ 
double phi(double x) {
    return std::tan(0.5 * std::exp(-x * x));
}

//  $\phi'(x) = -x * e^{-x^2} * \sec^2(e^{-x^2}/2)$ 
double dphi(double x) {
    double a = 0.5 * std::exp(-x * x);
    double c = std::cos(a);
    double sec2 = 1.0 / (c * c);
    return -x * std::exp(-x * x) * sec2;
}

// Авто-отделение корня: ищем смену знака на сетке
bool find_bracket(double x_min, double x_max, double step, double &Left,
double &Right) {
    double prev = x_min;
    double fprev = f(prev);

    for (double x = x_min + step; x <= x_max + 1e-15; x += step) {
        double fx = f(x);

        if (fprev == 0.0) { Left = prev; Right = prev; return true; }
        if (fprev * fx < 0.0) { Left = prev; Right = x; return true; }

        prev = x;
        fprev = fx;
    }
    return false;
}

int main() {
    std::cout << std::fixed << std::setprecision(12);

    // 1) Отделяем корень
    double Left = 0.0, Right = 0.0;
```

```

    if (!find_bracket(-5.0, 5.0, 0.1, Left, Right)) {
        std::cerr << "Не удалось отделить корень на [-5;5] с шагом 0.1\n";
        return 1;
    }

    std::cout << "Отделение корня (авто): [" << Left << "; " << Right <<
"]\n";
    std::cout << "f(Left)=" << f(Left) << ", f(Right)=" << f(Right) <<
"\n\n";

    // 2) Проверяем отображение интервала в себя и условие сходимости
    double phiL = phi(Left);
    double phiR = phi(Right);
    double q = estimate_q(dphi, Left, Right, 5000, 0.0);

    std::cout << "Выбрана функция phi(x) = tan(exp(-x^2)/2)\n";
    std::cout << "phi(Left)=" << phiL << ", phi(Right)=" << phiR << "\n";
    std::cout << "q = max |phi'(x)| на [Left;Right] = " << q << "\n";
    std::cout << "Так как q < 1, метод простых итераций сходится.\n\n";

    // 3) Выбор x0: любая точка из [Left;Right], берем середину
    double x0 = 0.5 * (Left + Right);
    std::cout << "Выбрано начальное приближение x0=" << x0 << "\n\n";

    // 4) Опорное значение корня (эталон): Delta=0, очень высокая точность
    auto ref = ITER(phi, dphi, Left, Right, x0, 1e-14, 1000000, 0.0);
    if (!ref.ok) {
        std::cerr << "Не удалось получить reference-корень методом простых
итераций\n";
        return 2;
    }
    std::cout << "Reference root (Eps=1e-14, Delta=0): x=" << ref.x
        << "   iters=" << ref.iters << "\n";
    std::cout << "Проверка f(x_ref)=" << f(ref.x) << "\n\n";

    // 5) Скорость сходимости: Iterations vs Eps (0.1 .. 1e-10)
    std::ofstream feps("iterations_vs_eps_iter.csv");
    feps << "Eps;Iterations;Root;LastStep\n";

    for (int i = 0; i <= 90; ++i) {
        double t = i / 90.0;           // 0..1
        double p = -1.0 - 9.0 * t;     // -1..-10
        double Eps = std::pow(10.0, p);

        auto r = ITER(phi, dphi, Left, Right, x0, Eps, 1000000, 0.0);

        feps << std::setprecision(16)
            << Eps << ";" << r.iters << ";" << r.x << ";" << r.last_step
<< "\n";
    }
    feps.close();
    std::cout << "CSV сохранён: iterations_vs_eps_iter.csv\n";

    // 6) Обусловленность: влияние Delta (округление phi и phi')
    const double EpsFix = 1e-6;
    std::vector<double> deltas = {0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-
6, 1e-7, 1e-8};

```

```

std::ofstream fdel("sensitivity_vs_delta_iter.csv");
fdel << "Delta;Eps;Iterations;Root;AbsErrorToRef\n";

for (double Delta : deltas) {
    auto r = ITER(phi, dphi, Left, Right, x0, EpsFix, 1000000, Delta);
    double err = std::abs(r.x - ref.x);

    fdel << std::setprecision(16)
        << Delta << ";" << EpsFix << ";" << r.iters << ";" << r.x <<
";" << err << "\n";
}
fdel.close();
std::cout << "CSV сохранён: sensitivity_vs_delta_iter.csv\n\n";

// 7) Один рабочий запуск для вывода результата в консоль
const double EpsWork = 1e-8;
const double DeltaWork = 1e-10;
auto work = ITER(phi, dphi, Left, Right, x0, EpsWork, 1000000,
DeltaWork);

std::cout << "Рабочий запуск: Eps=" << EpsWork << ", Delta=" <<
DeltaWork << "\n";
std::cout << "Корень x=" << work.x << "\n";
std::cout << "f(x)=" << f(work.x) << "\n";
std::cout << "Итераций=" << work.iters << "\n";
std::cout << "Последний шаг=" << work.last_step << "\n";
std::cout << "Абсолютная ошибка относительно эталона=" <<
std::abs(work.x - ref.x) << "\n";

// Так как  $\phi'(x) < 0$  на  $[Left; Right]$ , по методичке погрешность не
превышает Eps
double sign_check = dphi(0.5 * (Left + Right));
if (sign_check < 0.0) {
    std::cout << "Так как  $\phi'(x) < 0$  на интервале, теоретическая
оценка погрешности:  $\leq Eps$ \n";
} else {
    std::cout << "Теоретическая оценка погрешности:  $q \cdot Eps / (1-q) =$ 
<< q * EpsWork / (1.0 - q) << "\n";
}

return 0;
}

```

Название файла: plot_graphs.py

```

import pandas as pd
import matplotlib.pyplot as plt

# 1) Iterations vs Eps
eps_df = pd.read_csv("iterations_vs_eps_iter.csv", sep=";")

plt.figure()
plt.plot(eps_df["Eps"], eps_df["Iterations"], marker="o", linewidth=1)
plt.xscale("log")
plt.xlabel("Eps")
plt.ylabel("Iterations")
plt.title("Simple Iterations: Iterations vs Eps")

```

```

plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("iterations_vs_eps_iter.png", dpi=200)
plt.show()

# 2) Error vs Delta (Delta=0 убираем для log по X)
delta_df = pd.read_csv("sensitivity_vs_delta_iter.csv", sep=";")
delta_df_nonzero = delta_df[delta_df["Delta"] > 0].copy()

plt.figure()
plt.plot(delta_df_nonzero["Delta"], delta_df_nonzero["AbsErrorToRef"],
marker="o", linewidth=1)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Delta")
plt.ylabel("|x_delta - x_ref|")
plt.title("Simple Iterations: Sensitivity (Error vs Delta)")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("sensitivity_vs_delta_iter.png", dpi=200)
plt.show()

```